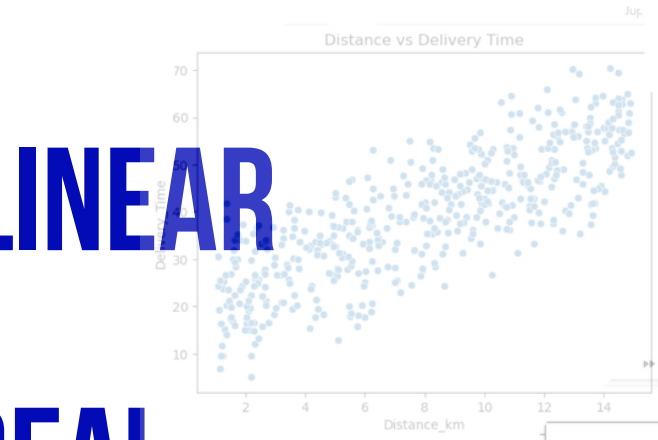
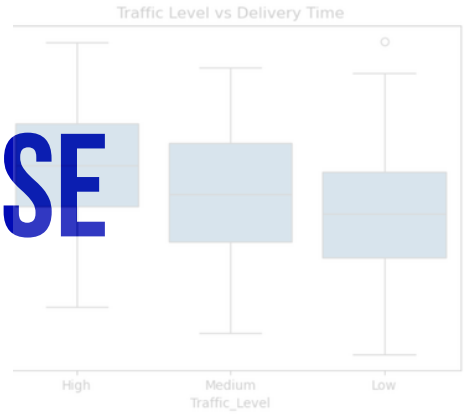
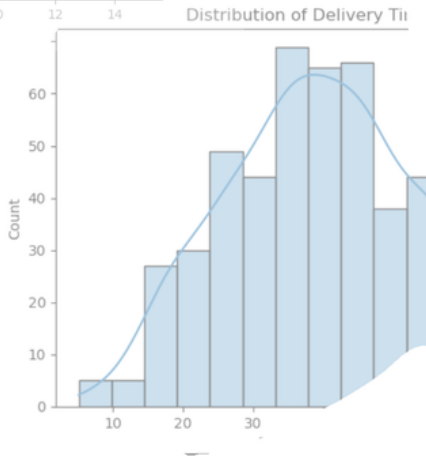
NISHA SABALE

MASTERING LINEAR REGRESSION THROUGH A REAL-WORLD FOOD DELIVERY CASE STUDY







Why Are We Learning Linear & Logistic Regression in ML?

Because every ML problem starts with 2 questions:
1.Are we predicting a number?
2.Or predicting a category?

OBJECTIVES

- Predict Delivery Time (in minutes) using Linear Regression
- Understand how different factors like distance, traffic, preparation time, and rider experience impact delivery time
- Build a model that helps in improving delivery efficiency and customer satisfaction
- Generate actionable insights for business decision-making

APPROACH

- Define the business problem and identify target variable (Delivery_Time)
- Perform Exploratory Data Analysis (EDA) to understand relationships between variables
- Apply data preprocessing (encoding categorical variables, feature selection)
- Split data into training and testing sets
- Build Linear Regression model (simple → multiple)
- Evaluate model using MAE, MSE, RMSE, and R^2 score
- Analyze residuals to validate model assumptions
- Interpret coefficients to derive business insights

PREREQUISITE STEPS

1. Understand the Problem

Identify problem type → Regression (continuous output)
Target Variable → Delivery_Time

2. Understand the Data

Check features → Distance, Traffic, Weather, Prep Time, etc.
Understand data types (numerical & categorical)

3. Data Cleaning

Ensure no missing or inconsistent values
Remove irrelevant columns (if any)

4. Data Preparation

Convert categorical → numerical (Encoding)
Select important features
Scale data (if required)

5. Basic Data Analysis

Check relationships between variables
Example:
Delivery Time vs Distance
Delivery Time vs Traffic
Delivery Time vs Preparation Time

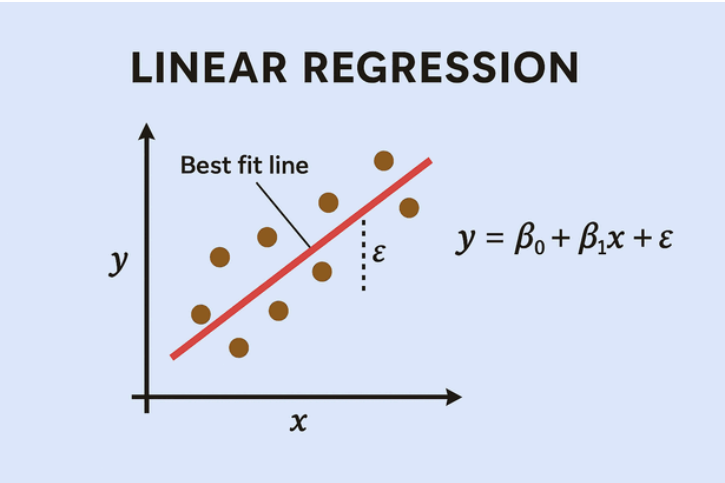
6. Choose Model

Linear Regression → Predict delivery time (continuous output)

7. Train & Test Split

Split dataset into training and testing data
Train model on training data
Evaluate on testing data

Linear Regression Intuition :



Imagine you are working in a company where you need to predict the price of houses based on features like area, number of rooms, and location.

The output here is a number (continuous value) — not a category.
To solve this type of problem, we need a model that can understand the relationship between input features and a numeric outcome.

That's why we learn Linear Regression —
because it helps us find a relationship and predict continuous values accurately.

Linear Regression is a fundamental supervised learning algorithm used to model the relationship between a dependent variable and one or more independent variables. It predicts continuous values by fitting a straight line that best represents the data.

What Linear Regression Does Here

Linear Regression tries to find the relationship between delivery-related factors and delivery time

Example Insights:

- Higher distance → Higher delivery time
- High traffic → More delay
- More preparation time → Longer delivery

Instead of guessing, the model learns these relationships from data

1. Best-Fit Line (Core Idea)

The model draws a line (or a plane when multiple features are used) that:

- Best represents the relationship between inputs and delivery time
- Minimizes prediction error
- Helps predict delivery time for new orders

In real-world terms:

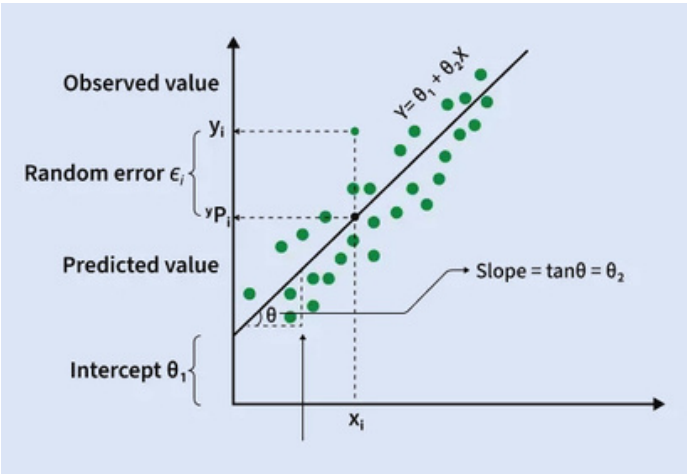
"Given order details → estimate delivery time accurately"

Understanding Variables

- Y (Dependent / Target Variable) → Delivery_Time
- X (Independent Variables / Predictors) →
  - Distance_km
  - Traffic_Level
  - Preparation_Time
  - Rider_Experience
  - Weather, etc.

Goal:

Predict Y using X



2. Equation of the Best-Fit Line

For simple linear regression (with one independent variable), the best-fit line is represented by the equation - y=mx+b

- y → Predicted Delivery Time
- x → Feature (e.g., Distance_km)
- m → Impact of that feature (slope)
- b → Base delivery time (intercept)

3. Interpretation

Suppose:

- m = 2.5 for Distance_km

That means:

- If distance increases by 1 km,
- delivery time increases by 2.5 minutes

In Multiple Linear Regression:

- Each feature has its own coefficient
- Model learns:
  - Which factor affects delivery time most
  - How strongly it impacts

4. Error & Residuals

To find the best-fit line, we use a method called <u>Least Squares</u>. The idea behind this method is to minimize the sum of squared differences between the actual values (data points) and the predicted values from the line. These differences are called residuals.

Example:

- Actual Delivery Time = 40 minutes
- Predicted Delivery Time = 35 minutes

Error (Residual) = 5 minutes

$$\Sigma(y_i - \hat{y}_i)^2$$

Assumptions of the Linear Regression

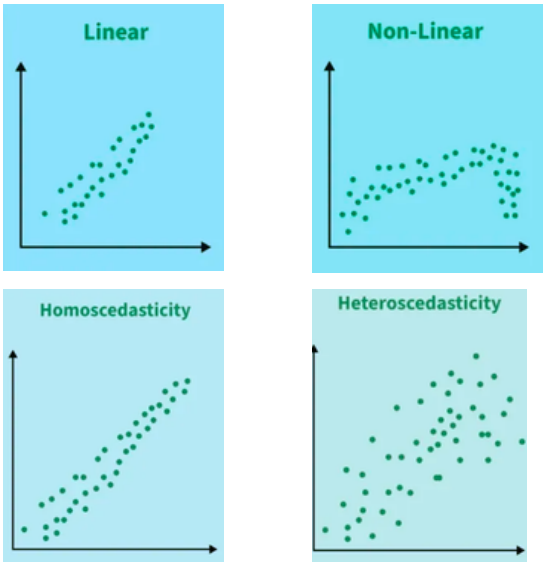
1. Linearity: The relationship between inputs (X) and the output (Y) is a straight line.

2. Independence of Errors: The errors in predictions should not affect each other

3. Constant Variance (Homoscedasticity) :In Linear Regression, one important assumption is that the errors (residuals) should have a constant spread across all values of the input variable

4. Normality of Errors :Errors should follow a bell-shaped (normal) distribution Means most errors are small, few are large

5. No Multicollinearity
Input features should not be highly correlated with each other. Example: Age & Experience (both giving same info)



5. Model Evaluation Metrics (How do we know model is good?)

R Square / Adjusted R Square:
R^2 measures how well the model explains the variation in the target variable, with values between 0 and 1.
A value closer to 1 means the model fits the data well.
However, R^2 always increases when more features are added, even if they are not useful.
Adjusted R^2 solves this by penalizing unnecessary features and giving a more realistic score.
So, Adjusted R^2 is preferred when comparing models with multiple variables to avoid overfitting.

1. Top part (Numerator): → Error between actual delivery time and predicted delivery time "How wrong is my model?"
2. Bottom part (Denominator)→ Difference between actual value and average delivery time "How much data varies normally?"

R^2 = 1 - (Model Error / Total Variation)

- If model error is small → R^2 is high (close to 1)
- If model error is large → R^2 is low (close to 0)

$$R^2 = 1 - \frac{SS_{Regression}}{SS_{Total}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Mean Square Error(MSE)/Root Mean Square Error(RMSE):

MSE (Mean Squared Error) measures the average of squared differences between actual and predicted values, giving an absolute error value.
RMSE is simply the square root of MSE, which brings the error back to the same unit as the target (e.g., minutes).
Unlike R^2 (which is a relative measure), MSE/RMSE tell us the actual size of the error.
Errors are squared to avoid negative values and to penalize large mistakes more heavily.
MSE can look very large because of squaring, while RMSE is easier to interpret in real-world terms.

- y → Actual value
- ŷ (y hat) → Predicted value
- (y - ŷ) → Error (difference)
- Square (²) → Removes negative values & penalizes large errors
- 1/n → Average of all errors

So, MSE = average of squared errors
Same as MSE, but we take square root at the end Which is **RMSE**

$$MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

Mean Absolute Error (MAE):
MAE (Mean Absolute Error) is similar to MSE, but instead of squaring errors, it takes the absolute difference, so all errors are positive and easy to interpret. Unlike MSE, we don't square or take root, so MAE directly gives error in actual units (minutes).

- This makes MAE very intuitive: e.g., MAE = 5 means predictions are off by 5 minutes on average.
- Residuals are the differences between actual and predicted values, used to check model performance visually.
- If residual plots show a random scatter (no pattern) or a bell-shaped distribution, it indicates the model is performing well and predictions are close to real values.

- y → Actual value (real delivery time)
- ŷ (y hat) → Predicted value from model
- |y - ŷ| → Absolute error (difference, always positive)
- Σ (summation) → Add all errors
- n → Total number of data points

$$MAE = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i|$$

6. Types of Linear Regression:

6.1. Basic Types (Variable-Based):

1. Simple Linear Regression
Simple linear regression is the most basic form of linear regression, involving only one independent variable and one dependent variable. For example, in our food delivery case study, we can study the relationship between distance (independent variable) and delivery time (dependent variable).

$$Y = a + bX$$

where:
- Y is the dependent variable (Delivery Time)
- X is the independent variable (Distance_km)
- a is the intercept (base delivery time when distance is zero)
- b is the slope (how much delivery time increases for each additional kilometer)

Simple linear regression aims to find the best values for 'a' and 'b' to create the best-fit line. This line helps us predict delivery time based only on distance. However, in real-world scenarios, delivery time depends on multiple factors, so this model may not be very accurate.

2. Multiple Linear Regression
Multiple linear regression is an extension of simple linear regression and is used when more than one independent variable is involved. In our case study, delivery time depends on multiple factors such as distance, traffic level, preparation time, rider experience, and weather conditions.
In this case, the equation becomes:

$$Y=a+b1X1+b2X2+b3X3+...Y$$

where:
- Y is the dependent variable (Delivery Time)
- X_1, X_2, X_3... are independent variables (Distance, Traffic, Prep Time, etc.)
- a is the intercept (base delivery time)
- b_1, b_2, b_3... are coefficients showing impact of each feature

Each coefficient represents how much the delivery time changes when that specific feature changes, keeping other variables constant.
In multiple linear regression, the goal is to find the best values for all coefficients so that the model fits the data well. This allows us to make more accurate predictions and understand which factors impact delivery time the most.
When building the model, it is important to carefully choose relevant variables, because including unnecessary features can reduce model performance or lead to overfitting.

3. Polynomial Regression
Polynomial regression is an extension of linear regression used when the relationship between the independent and dependent variables is not strictly linear. In real-world scenarios, the effect of a feature may not always increase in a straight line. For example, in our food delivery case study, delivery time may increase slowly at shorter distances but rise much faster after a certain distance due to traffic or route complexity.
In polynomial regression, we model this non-linear relationship by adding higher-degree terms of the independent variable, **such as:**

$$Y=a+b1X+b2X2+b3X3+...Y$$

where:
- Y is the dependent variable (Delivery Time)
- X is the independent variable (e.g., Distance_km)
- a is the intercept
- b_1, b_2, b_3... are coefficients of different degree terms

These additional terms allow the model to capture curved relationships instead of just straight lines.
Even though the relationship looks non-linear, it is still called "linear regression" because the model is linear in terms of its coefficients.
Polynomial regression helps improve model performance when simple linear models fail to capture complex patterns. However, using very high-degree polynomials can lead to overfitting, so it should be used carefully.

6.2. Regularized Regression (Overfitting Prevention):

Regularized regression is an extension of linear regression that is used to prevent overfitting and handle issues like multicollinearity (highly correlated features). In complex models with many variables, the model may start learning noise instead of actual patterns. To solve this, regularization techniques add a penalty term to the standard linear regression equation. These penalties reduce the size of coefficients, making the model more generalizable.

7.1. Lasso Regression (L1 Regularization)

Lasso Regression adds a penalty equal to the absolute value of coefficients:

- $\sum(Y - \hat{Y})^2 \rightarrow$ Normal error (same as Linear Regression)
- $\lambda \sum|b_i| \rightarrow$ Penalty term (absolute values of coefficients)

$$\text{Cost} = \sum(Y - \hat{Y})^2 + \lambda \sum |b_i|$$

What it does:

- Adds penalty based on absolute value of coefficients
- Forces some coefficients to become exactly zero

Key Points:

- Can shrink some coefficients to exactly zero
- Performs automatic feature selection
- Helps remove less important variables

In Your Case Study:

If Weather has very little impact, Lasso can remove it completely from the model.

7.2. Ridge Regression (L2 Regularization):

Ridge Regression adds a penalty equal to the square of the coefficients:

where:

- $\sum(Y - \hat{Y})^2 \rightarrow$ Normal error
- $\lambda \sum b_i^2 \rightarrow$ Penalty term (square of coefficients)

$$\text{Cost} = \sum(Y - \hat{Y})^2 + \lambda \sum b_i^2$$

What it does:

- Penalizes large coefficients more strongly
- Shrinks coefficients closer to zero but NOT exactly zero

In Your Case Study:

If Distance and Preparation_Time are correlated, Ridge helps stabilize the model.

Now that we have understood the complete theory behind Linear Regression through this real-world food delivery case study, we will move to the practical implementation. In the attached Jupyter Notebook, you will see how each concept—data preprocessing, feature selection, model building, and evaluation—is applied step by step. This will help you clearly visualize how a regression model learns relationships between variables and predicts delivery time. You will also observe how model performance is evaluated using different metrics and how residuals are analyzed. This hands-on implementation will bridge the gap between theory and real-world application. By the end, you will have a clear understanding of how Linear Regression works in an industry scenario.

Hello, this is Nisha!

Welcometothis **End-to-EndLinearRegression Case Study**

In this notebook, I have covered Linear Regression from **basic concepts to advanced implementation** using a real-world **Food Delivery problem statement**.
This content is designed in a way that helps you:

- Understand concepts step-by-step
- Apply them practically using Python
- Build strong clarity for interviews

All the code is written with proper explanations so that even beginners can follow along easily. If you face any difficulty while understanding any step, go through the explanations carefully—you will get clarity.

My goal is simple:
To help you not just learn Linear Regression, but confidently apply and explain it.
Let's get started!

Problem Statement -

In the food delivery industry, accurately predicting delivery time is crucial for improving customer satisfaction and operational efficiency. Delivery time depends on multiple factors such as distance between the restaurant and customer, traffic conditions, weather, preparation time, and rider experience. Due to the variability of these factors, it becomes difficult to estimate delivery time manually.

The objective of this case study is to build a Linear Regression model that can predict the delivery time of an order based on these influencing factors. By understanding the relationship between input features and delivery time, we aim to generate insights that help optimize delivery operations and reduce delays.

step 1. Import data

In [1]:
```
#Import required libraries

import pandas as pd          # for data handling
import numpy as np           # for numerical operations
import matplotlib.pyplot as plt   # for visualization
import seaborn as sns        # for advanced visualization
```

In [2]:
```
# Load your dataset

df = pd.read_csv("food_delivery_dataset.csv")
```

```
# Display first 5 rows
df.head()
```

Out[2]:

	Distance_km	Order_Value	Traffic_Level	Weather	Rider_Experience	Restaurant_Ratin
0	6.24	728.35	High	Rainy	9.9	3
1	14.31	582.49	Medium	Stormy	8.6	4
2	11.25	378.57	High	Rainy	2.5	4
3	9.38	832.42	High	Clear	9.3	2
4	3.18	716.26	Low	Clear	1.6	4

Step 2 – Insights

In [4]:

```
# Check number of rows and columns
df.shape
```

Out[4]:

```
(500, 10)
```

In [5]:

```
# Check column names
df.columns
```

Out[5]:

```
Index(['Distance_km', 'Order_Value', 'Traffic_Level', 'Weather',
       'Rider_Experience', 'Restaurant_Rating', 'Preparation_Time',
       'Time_of_Day', 'Weekend', 'Delivery_Time'],
      dtype='object')
```

In [6]:

```
# Check data types and null values
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 10 columns):
 #   Column             Non-Null Count Dtype
---  ------             -------------- -----
 0   Distance_km        500 non-null   float64
 1   Order_Value        500 non-null   float64
 2   Traffic_Level      500 non-null   object
 3   Weather            500 non-null   object
 4   Rider_Experience   500 non-null   float64
 5   Restaurant_Rating  500 non-null   float64
 6   Preparation_Time   500 non-null   float64
 7   Time_of_Day        500 non-null   object
 8   Weekend            500 non-null   int64
 9   Delivery_Time      500 non-null   float64
dtypes: float64(6), int64(1), object(3)
memory usage: 39.2+ KB
```

In [7]:

```
# Statistical summary
df.describe()
```

Out[7]:

	Distance_km	Order_Value	Rider_Experience	Restaurant_Rating	Preparation_Time
count	500.000000	500.000000	500.000000	500.000000	500.000000
mean	7.979820	533.756500	5.273600	3.746600	25.007000
std	4.181815	256.944181	2.755949	0.722643	8.753483
min	1.070000	104.170000	0.500000	2.500000	10.000000
25%	4.380000	306.192500	2.800000	3.200000	17.800000
50%	8.185000	524.640000	5.300000	3.700000	25.050000
75%	11.587500	753.705000	7.700000	4.400000	32.325000
max	14.900000	999.750000	10.000000	5.000000	39.800000

Understanding the Data (Step 2 – Insights) After loading and exploring the dataset, we gain a clear understanding of the structure
and components of the data.
The dataset contains multiple features that influence the delivery time, including both numerical variables (such as distance, order value, preparation time, rider experience) and categorical variables (such as traffic level, weather conditions, and time of day). This indicates that before building the model, we will need to perform preprocessing steps like encoding categorical variables. The target variable in this dataset is Delivery_Time, which is a continuous numerical value.
This confirms that the problem is a regression problem, where the goal is to predict a numeric outcome. From the statistical summary, we can observe the range, mean, and distribution of
numerical features. This helps us understand whether the data is properly scaled and whether there are any unusual values. Since the dataset is already clean, we do not need to handle missing values, which simplifies the preprocessing step.
Additionally, by looking at the features, we can infer that delivery time is influenced by multiple real-world factors such as distance, traffic, and preparation time. This suggests that Multiple Linear Regression will be more appropriate than Simple Linear Regression for this problem.
Overall, this step helps us clearly identify: The type of problem (Regression) The target variable (Delivery_Time) The input features
(multiple influencing factors) The need for preprocessing (especially encoding categorical variables) “Understanding the data is a crucial step before modeling, as it helps in selecting the
right approach and avoiding mistakes in later stages.”

Step 3: Identify Target & Features (Explanation)

Inthis step, we clearlydefinewhatwewanttopredictandwhichvariables we will use to make that prediction.

The target variable (also called the dependent variable) is the output we want our model to predict. In this case, the target variable is Delivery_Time, which represents the total time taken to deliver an order. Since this is a continuous numerical value, it confirms that we are solving a regression problem.
The feature variables (also called independent variables) are the inputs that influence the target variable. These include factors such as distance, traffic level, preparation time, rider experience, weather conditions, and time of day. These features provide the necessary information for the model to learn patterns and relationships.
Separating the dataset into features (X) and target (y) is an important step before model building. It ensures that the model learns only from the input variables and does not use the output variable during training.

This step is also important because it helps us:

Clearly understand what we are predicting Identify which variables are useful for prediction Prepare the data for model training
Simple Understanding

Target (y) → What we want to predict (Delivery Time) Features (X) → Factors affecting prediction (Distance, Traffic, etc.)
“Defining target and features correctly is crucial, as it directly impacts how well the model learns and performs"

In [8]:
```
# Target variable (what we want to predict)
target = "Delivery_Time"

# Features (input variables)
features = [
    "Distance_km",
    "Order_Value",
    "Traffic_Level",
    "Weather",
    "Rider_Experience",
    "Restaurant_Rating",
    "Preparation_Time",
    "Time_of_Day",
    "Weekend"
]

print("Target:", target)
print("Features:", features)
```

```
Target: Delivery_Time
Features: ['Distance_km', 'Order_Value', 'Traffic_Level', 'Weather', 'Rider_Exper
ience', 'Restaurant_Rating', 'Preparation_Time', 'Time_of_Day', 'Weekend']
```

STEP 4: Basic Data Analysis (EDA)

In [9]:
```
#4.1 Check Distribution of Target
# Distribution of Delivery Time

sns.histplot(df["Delivery_Time"], kde=True)
plt.title("Distribution of Delivery Time")
plt.show()
```

How to Read above Plot 1 Center of Data (Peak) =The highest bars / peak of curve show: Most common delivery time range

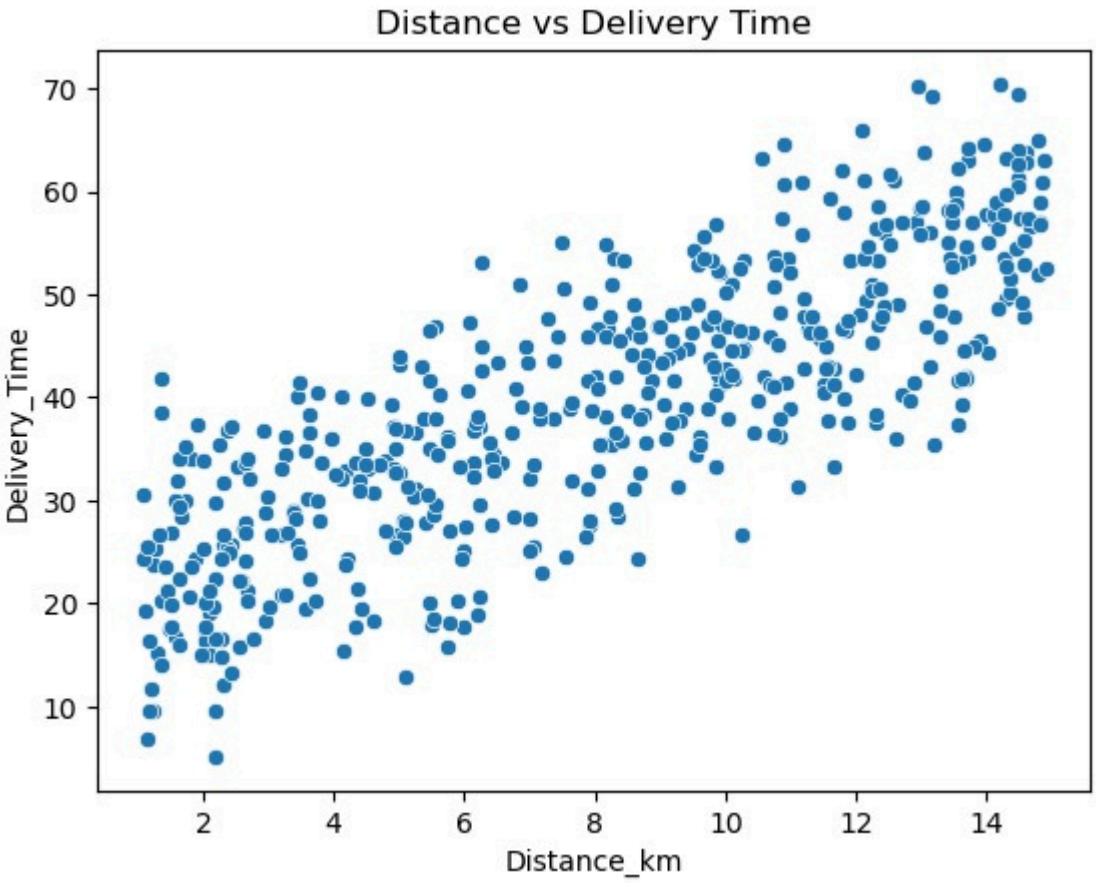
Example: If peak is around 30–40 minutes → most orders are delivered in this range

2 Spread of Data How wide the graph is tells: Variation in delivery time Narrow → consistent delivery time Wide → high variability (good learning insight)
3 Shape of Distribution Ideally (for Linear Regression):

Should look like a bell-shaped curve (normal distribution) .Possible Shapes & Meaning: Bell Shape (Normal Distribution) → Good for Linear Regression Right Skewed (tail on right) → Some deliveries take much longer (delays) Left Skewed → Few unusually fast deliveries 4 Outliers : Very high or low values (extreme bars)Indicate unusual delivery times

4.2 Relationship: Distance vs Delivery Time

In [10]:
```
sns.scatterplot(x="Distance_km", y="Delivery_Time", data=df)
plt.title("Distance vs Delivery Time")
plt.show()
```



What this Above graph shows

Each dot = one order X-axis → Distance (km) Y-axis → Delivery Time (minutes)

It shows the relationship between distance and delivery time

What to Observe in This Plot

Relationship (MOST IMPORTANT) Check direction of points:If points go upward (↗) → Positive relationship

Meaning:More distance → More delivery time

2. Linearity (Important for Linear Regression) =Check if points roughly form a straight-line pattern Straight trend → Linear relationship (good for model) Curved pattern → Non-linear (may need polynomial)

3. Strength of Relationship

Look how tightly points are grouped:

Close to line → Strong relationship Very scattered → Weak relationship

4. Spread (Variance) For same distance, are delivery times similar? Small spread → consistent delivery Large spread → other factors (traffic, prep time) affecting
5. Outliers

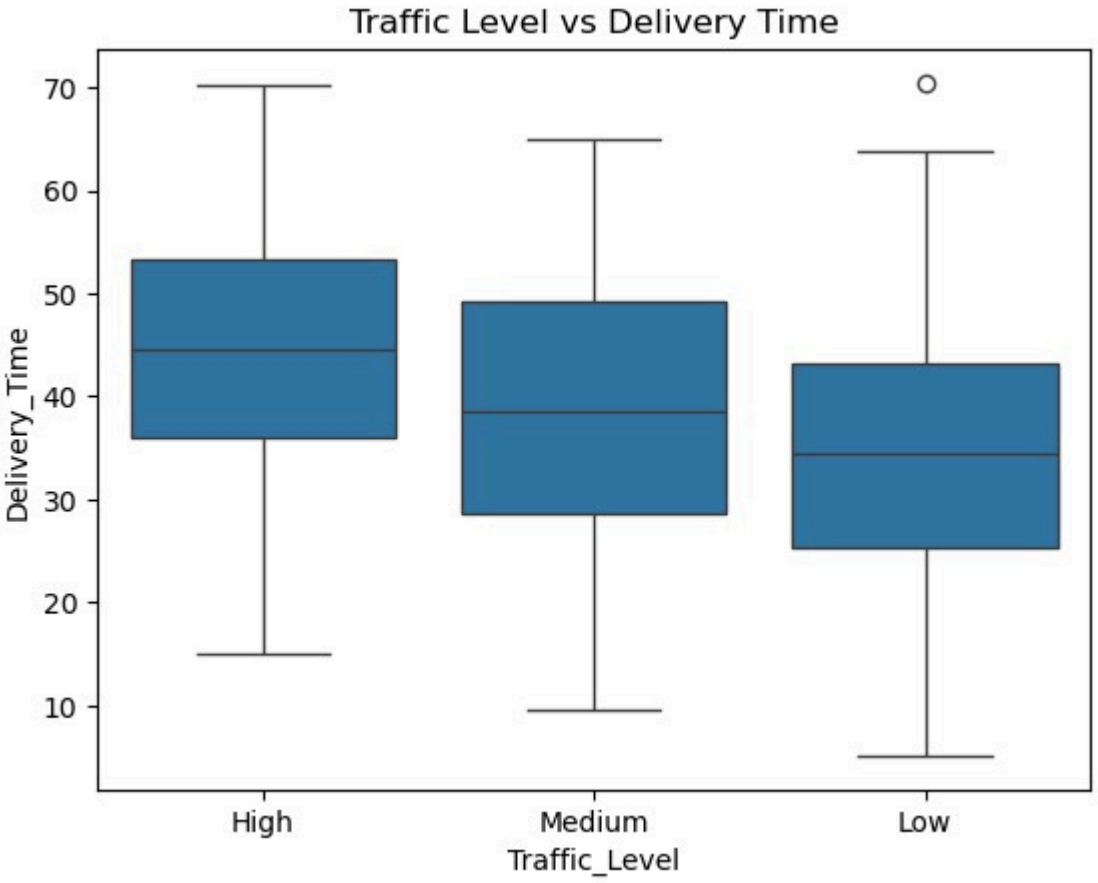
Points far away from pattern: Very high delivery time at low distance, Indicates unusual cases (delay, traffic issue)

Interpretation From this graph, we understand:

Distance is a key factor affecting delivery time .There is likely a positive linear relationship ,But delivery time is not only dependent on distance →Other features (traffic, prep time) also matter

4.3 Traffic Impact

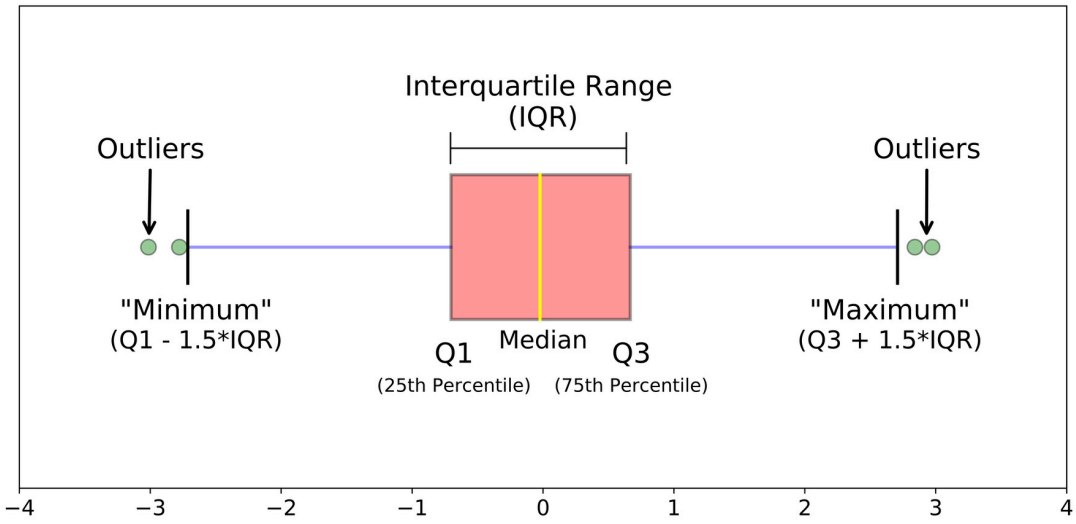
In [11]:
```
sns.boxplot(x="Traffic_Level", y="Delivery_Time", data=df)
plt.title("Traffic Level vs Delivery Time")
plt.show()
```



What is a Boxplot Explained below refer the image :

Aboxplotshowsthe distribution of data using5key values:

Components: Median (middle line in box) → Middle value of data Box (Q1 to Q3) → 50% of data lies here Lower line (min) → Lowest value (excluding outliers) Upper line (max) → Highest value (excluding outliers) Dots outside → Outliers (unusual values)



What to Understand from This Graph

1.Compare Median (MOST IMPORTANT)= Look atmiddle lineineachbox:Lowtraffic → lower median Medium traffic → higher High traffic → highest

Meaning:Delivery time increases with traffic level

2. Compare Spread (Box Size) = Bigger box = more variation High traffic → larger box Delivery time is less predictable in heavy traffic 3.Compare Range (Whiskers) =Longer lines = more variation High traffic → longer range ,Some deliveries take much longer

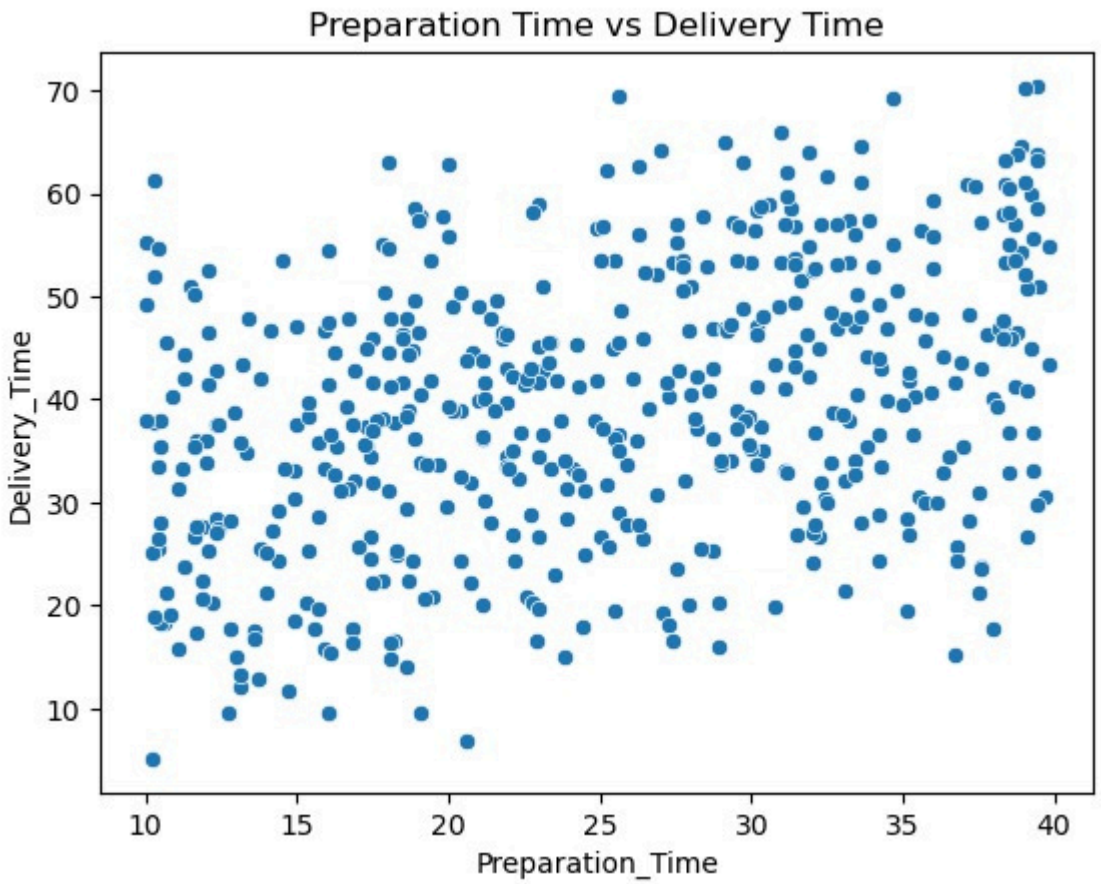
4. Outliers: Dots outside box:Very high delivery times .Shows extreme delays (maybe traffic jams, weather issues) final Interpretation for Your Case Study

From this plot, we understand:

Trafficlevelhas astrongimpact ondeliverytime .As trafficincreases → delivery time increases High traffic introduces more variability and delays

4.4 Preparation Time Impact

In [13]:
```
sns.scatterplot(x="Preparation_Time", y="Delivery_Time", data=df)
plt.title("Preparation Time vs Delivery Time")
plt.show()
```



What this graph shows : Each point represents: One order with its prep time and delivery time

What to Observe

1 Relationship (MOST IMPORTANT) Check direction of points:If points go upward (↗) :Positive relationship Meaning:More preparation time → More delivery time

2 Linearity

Check if points follow a straight-line trend:Rough straight pattern → Good for Linear Regression Curve → Non-linear In this case: Likely linear trend exists
3 Strength of Relationship How close points are:Close grouping → strong relationship Scattered → weaker Usually:Prep time shows moderate to strong relationship
4 Spread (Important Insight) For same preparation time:Delivery time may vary Meaning:Other factors (traffic, distance) also affect delivery
5 Outliers Points far away:Very high delivery time with low prep time Indicates:External delays (traffic, weather)
From this graph, we understand:

Preparation time has a direct positive impact on delivery time .It is an important predictor variable But it is not the only factor → multiple variables influence delivery

STEP 5: Data Preprocessing

5.1 Convert Categorical → Numerical

In [14]:
```
# Convert categorical variables using one-hot encoding because Machine Learning
#(like Linear Regression) cannot understand text values

df_encoded = pd.get_dummies(df, drop_first=True)

df_encoded.head()
```

Out[14]:

	Distance_km	Order_Value	Rider_Experience	Restaurant_Rating	Preparation_Time W
0	6.24	728.35	9.9	3.7	28.
1	14.31	582.49	8.6	4.9	2
2	11.25	378.57	2.5	4.1	18.
3	9.38	832.42	9.3	2.8	9
4	3.18	716.26	1.6	4.2	14.

1

29.

5

STEP 6: Split Data (Train-Test)

In [15]:
```
from sklearn.model_selection import train_test_split

# Define X (features) and y (target)
X = df_encoded.drop("Delivery_Time", axis=1)
y = df_encoded["Delivery_Time"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

# test_size=0.2
# 20% data → Testing
# 80% data → Training

# Example:

# Total = 500 rows
# Train = 400
# Test = 100
```

```
Training data: (400, 12)
Testing data: (100, 12)
```

Explantion : The dataset is split into training and testing sets to evaluate model performance on unseen data. The model is trained on 80% of the data and tested on 20%, ensuring that it generalizes well and does not simply memorize the training data.

STEP 7: Build Linear Regression Model

In [16]:
```
from sklearn.linear_model import LinearRegression

# Create model
model = LinearRegression()

# Train model
model.fit(X_train, y_train)
```

Out[16]:

```
▾ LinearRegression
▶ Parameters
```

STEP 8: Make Predictions

In [17]:

```
#Predict ontestdata
y_pred = model.predict(X_test)
```

STEP 9: Model Evaluation

In [18]:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# MAE
mae = mean_absolute_error(y_test, y_pred)

# MSE
mse = mean_squared_error(y_test, y_pred)

# RMSE
rmse = np.sqrt(mse)

# R2 Score
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
MAE: 1.843409777359962
MSE: 5.203494007597354
RMSE: 2.2811168333948513
R2 Score: 0.9669209686528784
```

STEP 10: Residual Analysis

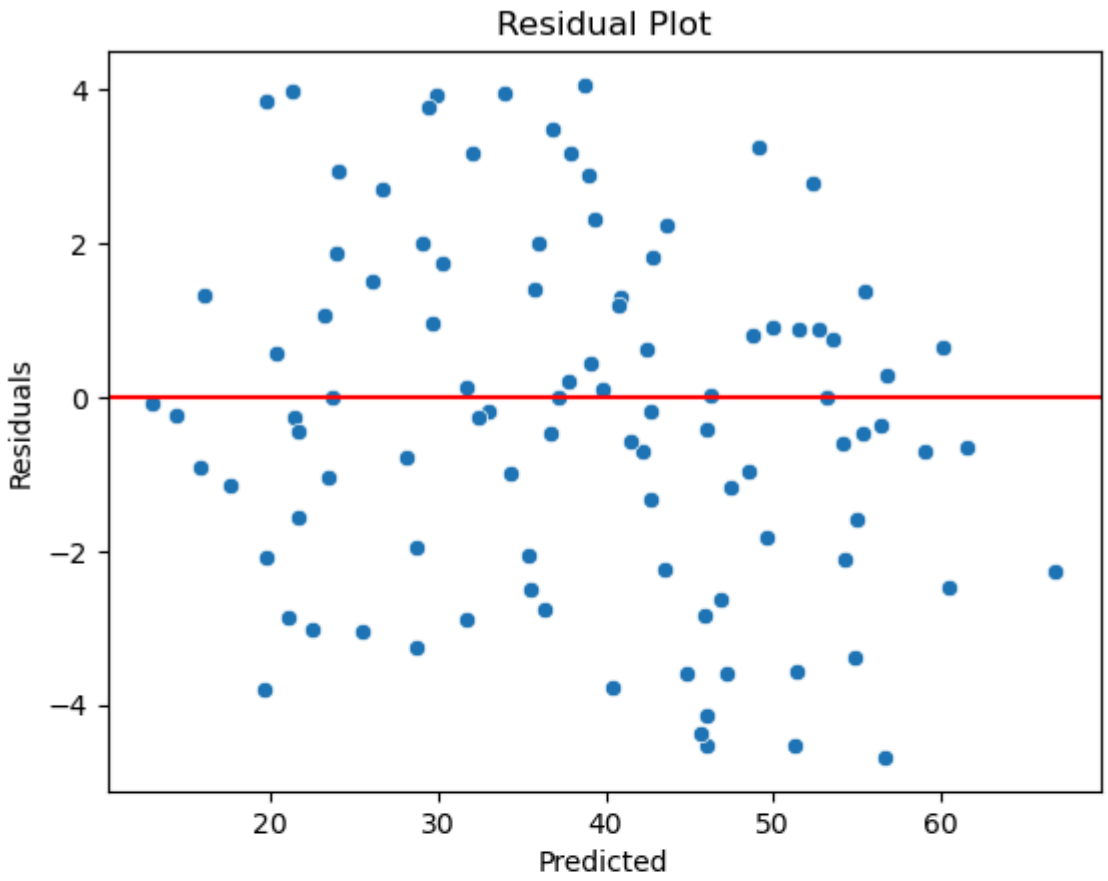
In [19]:

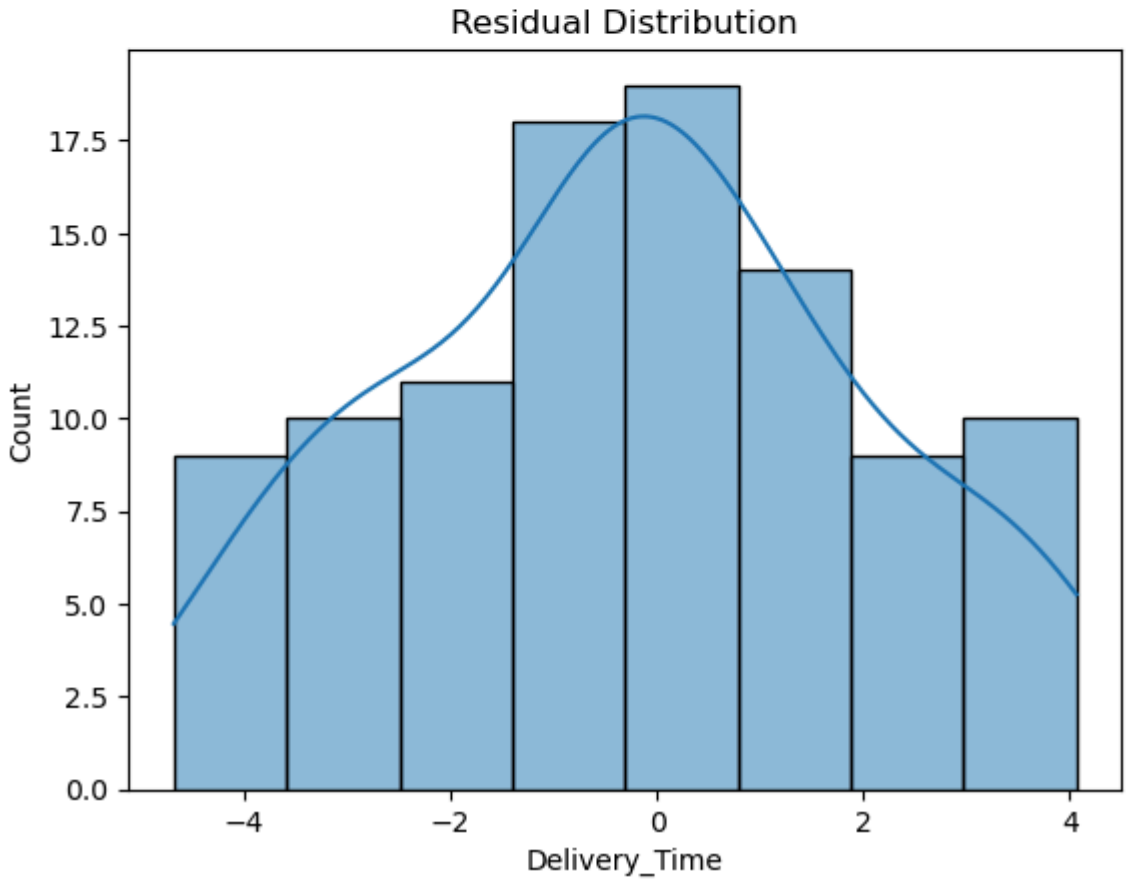
```
#Residuals
residuals = y_test - y_pred

#              Scatter              plot
sns.scatterplot(x=y_pred,  y=residuals)
plt.axhline(0,             color='red')
plt.title("Residual              Plot")
plt.xlabel("Predicted")
plt.ylabel("Residuals") plt.show()


# Distribution plot
sns.histplot(residuals, kde=True)
```

```
plt.title("Residual Distribution")
plt.show()
```





Residual analysis shows how well the model fits the data. A random scatter of residuals around zero indicates that the model captures the underlying pattern effectively. Additionally, a near-normal distribution of residuals suggests that model assumptions are satisfied and predictions are reliable .

“Residualplots helpvalidate model assumptions.Randomdistributionindicatesgoodfit, while patterns suggest model issues like non-linearity or heteroscedasticity.”

You just performed:

Data understanding EDA Preprocessing Model training Evaluation Residual checking

Step 11.Interpretation of Coefficients

In [20]:
```
#Get model coefficients
#Coefficient tells:
#“How much the target (Delivery Time) changes when a feature increases by 1 unit
coefficients = pd.DataFrame({
"Feature": X.columns,
"Coefficient": model.coef_
})

coefficients.sort_values(by="Coefficient", ascending=False)
```

Out[20]:

	Feature	Coefficient
9	Weather_Stormy	6.903161
8	Weather_Rainy	3.704863
0	Distance_km	2.527420
4	Preparation_Time	0.491705
5	Weekend	0.013534
1	Order_Value	-0.000091
11	Time_of_Day_Night	-0.016054
3	Restaurant_Rating	-0.142783
10	Time_of_Day_Morning	-0.473371
2	Rider_Experience	-1.272154
7	Traffic_Level_Medium	-5.034870
6	Traffic_Level_Low	-10.060207

How to Interpret 1 Positive Coefficient Example: Distance_km = 2.5 Meaning: If distance increases by 1 km , Delivery time increases by 2.5 minutes

2 Negative Coefficient Example: Rider_Experience = -1.2 Meaning:More experienced rider ,Delivery time decreases
3 Magnitude (Size of Value) : Larger value = stronger impact 2.5 → strong effect 0.2 → small effect
Special Case: Encoded Variables Example: Traffic_Level_High = 5 Meaning:High traffic increases delivery time compared to Low traffic (reference category)
Understanding from This:

Model coefficients represent the impact of each feature on delivery time. A positive coefficient indicates an increase in delivery time, while a negative coefficient indicates a decrease. The magnitude of the coefficient reflects the strength of the relationship, helping identify the most influential features.

“Coefficients help interpret the model by showing the direction and strength of each feature’s impact on the target variable.”

Each coefficient tells: "How much Delivery Time changes when that feature increases by 1 unit (keeping others constant) | Feature | Coefficient | | ---------- | ---------- | | Distance_km | 2.5 | | Preparation_Time | 0.8 | | Rider_Experience | -1.2 | Distance_km = 2.5 If distance increases by 1 km → delivery time increases by 2.5 minutes Preparation_Time = 0.8 If prep time increases by 1 min → delivery time increases by 0.8 minutes Rider_Experience = -1.2 More experienced rider → reduces delivery time

steo 12. Ridge & Lasso Implementation

In [22]:
```
from sklearn.linear_model import Ridge

# Create Ridge model
ridge_model = Ridge(alpha=1.0)

# Train model
ridge_model.fit(X_train, y_train)

# Predictions
y_pred_ridge = ridge_model.predict(X_test)
```

In [23]:
```
from sklearn.metrics import r2_score, mean_squared_error

print("Ridge R2:", r2_score(y_test, y_pred_ridge))
print("Ridge RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_ridge)))
```

```
Ridge R2: 0.966898838039474
Ridge RMSE: 2.281879764956849
```

In [24]:
```
ridge_coeff = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": ridge_model.coef_
})

ridge_coeff.sort_values(by="Coefficient", ascending=False)
```

Out[24]:

	Feature	Coefficient
9	Weather_Stormy	6.777397
8	Weather_Rainy	3.596309
0	Distance_km	2.528804
4	Preparation_Time	0.491284
5	Weekend	0.005441
1	Order_Value	-0.000067
11	Time_of_Day_Night	-0.024316
3	Restaurant_Rating	-0.151147
10	Time_of_Day_Morning	-0.464375
2	Rider_Experience	-1.271251
7	Traffic_Level_Medium	-4.891417
6	Traffic_Level_Low	-9.872356

Lasso Regression (L1)

In [25]:
```
from sklearn.linear_model import Lasso

# Create Lasso model
lasso_model = Lasso(alpha=0.1)

# Train model
lasso_model.fit(X_train, y_train)

# Predictions
y_pred_lasso = lasso_model.predict(X_test)
```

In [26]:
```
print("Lasso R2:", r2_score(y_test, y_pred_lasso))
print("Lasso RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_lasso)))
```

```
Lasso R2: 0.965104373855371
Lasso RMSE: 2.3429155525658585
```

In [27]:
```
lasso_coeff = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": lasso_model.coef_
})

lasso_coeff.sort_values(by="Coefficient", ascending=False)
```

Out[27]:

	Feature	Coefficient
9	Weather_Stormy	6.047761
8	Weather_Rainy	2.799685
0	Distance_km	2.535993
4	Preparation_Time	0.489501
1	Order_Value	0.000021
3	Restaurant_Rating	-0.000000
5	Weekend	-0.000000
10	Time_of_Day_Morning	-0.000000
11	Time_of_Day_Night	0.000000
2	Rider_Experience	-1.251074
7	Traffic_Level_Medium	-4.151599
6	Traffic_Level_Low	-9.145957

Final Interpretation:

Ridge:Reduces impact of all features ,Helps with multicollinearity Lasso: Removes unnecessary features (coeff = 0) ,Performs feature selection

Final Conclusion

Inthis casestudy, wesuccessfullybuilt an end-to-end Linear Regression model to predict food delivery time based on multiple real-world factors such as distance, traffic conditions, preparation time, rider experience, and weather. Through Exploratory Data Analysis (EDA), we identified meaningful relationships between these variables and the target variable, which helped in selecting the right modeling approach.

We started with a basic Linear Regression model and evaluated its performance using metrics like MAE, RMSE, and R^2 score. The results showed that the model was able to capture a significant portion of the variation in delivery time. By interpreting the model coefficients, we gained valuable insights into how each factor impacts delivery time—for example, distance and preparation time increase delivery time, while rider experience can help reduce it.

To further improve model performance and handle potential issues like overfitting and multicollinearity, we applied **Ridge and Lasso Regression**. Ridge helped in stabilizing the model by reducing the impact of correlated features, while Lasso helped in identifying and removing less important features through feature selection.

Residual analysis confirmed that the model assumptions were reasonably satisfied, indicating a good fit between predicted and actual values. Overall, this project demonstrates how Linear Regression can be effectively used not only for prediction but also for extracting actionable business insights.

Business Impact

Helps estimate accurate delivery time for customers Improves operational planning and resource allocation Enhances customer satisfaction by reducing delays

Final Closing Line

“This case study highlights how data-driven modeling can transform raw data into meaningful insights, enabling better decision-making in real-world business scenarios.”